Metric-Grounded Agentic Text-to-SQL for Nested Event-Tracking Data: An Ablation Study with Open-Weight Models

Authors: [Tên tác giả], [Đơn vị] · **Corresponding author:** [email]
Target: Q3/Q4 journal (applied AI / information systems / data engineering). **Draft status:** bản nháp, cần các bước ở mục "Việc cần làm trước khi nộp" cuối file.

Abstract

Product-analytics data (mobile apps, games, e-commerce) is stored as event logs whose parameters are nested and heterogeneous, and the business concepts analysts ask about (daily active users, ARPPU, retention) are not columns in any schema. Text-to-SQL research has concentrated on flat relational benchmarks, and few studies isolate the contribution of an explicit metric (semantic) layer. We study an agentic Text-to-SQL pipeline consisting of ambiguity detection, metric resolution against a YAML semantic layer, schema linking, retrieval-based few-shot prompting, read-only execution and execution-feedback self-correction. We introduce EventSQL-Bench, a synthetic benchmark of 162 bilingual (Vietnamese/English) questions over a nested DuckDB event table, including 17 deliberately ambiguous questions and three question families withheld from the few-shot pool. Using an 8B open-weight model run locally, execution accuracy (EX) on 108 answerable test questions rises from 0.352 (single prompt) to 0.880 (few-shot), 0.963 (+ semantic layer) and 0.981 (+ self-correction). The semantic layer's gain over few-shot is statistically significant (exact McNemar $p = 0.0039$) and is largest on withheld families (3/12 → 10/12) and on the hardest level (0.64 → 0.91); self-correction alone is not significant. A lexical synonym resolver fails completely on 20 out-of-dictionary paraphrases (0/20), and an LLM fallback restores recall to 17/20 and end-to-end EX to 0.89. Unexpectedly, BM25 schema pruning lowers accuracy both on EventSQL-Bench (−0.056, $p = 0.031$) and on a 200-question BIRD sample (0.240 → 0.155). We discuss threats to validity, notably that gold SQL and semantic layer were written by the same author.

Keywords: Text-to-SQL; semantic layer; large language models; agentic workflow; event tracking; nested data; benchmark.

---

1. Introduction

Analysts in product teams routinely ask questions such as "what was ARPPU in February?" over event-tracking tables. These tables differ from the relational databases on which Text-to-SQL is usually evaluated in three ways. (i) **Nesting and heterogeneity:** each event type carries its own parameters, stored as arrays of key–value structs, so one column has different meanings depending on the event name. (ii) **Absent business semantics:** "revenue", "active user" and "conversion rate" are organisational definitions, not schema elements. (iii) **Ambiguity:** "revenue in January" may legitimately denote sales revenue, item revenue or ad revenue.

Large language models (LLMs) have raised accuracy on academic benchmarks substantially, yet the gap to realistic settings remains large: on Spider 2.0 an o1-preview-based agent solves 21.3% of tasks versus 91.2% on Spider 1.0 [R14]. Industrial systems address part of this gap by grounding generation in a semantic layer (e.g., Metabase's Metabot, dbt MetricFlow, Cube), but the research literature rarely measures what such a layer contributes in isolation.

Research questions.

- **RQ1.** Does an agentic pipeline (linking, retrieval, execution feedback) outperform a single prompt on nested event data?
- **RQ2.** What does an explicit metric layer add beyond few-shot prompting, particularly on unseen question types and under paraphrase?
- **RQ3.** Can a small open-weight model deliver usable accuracy locally, and how does it degrade with model size?

Contributions.

1. A gap analysis of the Text-to-SQL literature for event-tracking data (Section 2).
2. EventSQL-Bench, a reproducible benchmark with nested parameters, metric-labelled questions, ambiguous questions, withheld question families and out-of-dictionary paraphrases.
3. A controlled ablation with paired significance tests that separates the effects of linking, few-shot retrieval, the semantic layer, self-correction and clarification.
4. Negative and cautionary findings: BM25 schema pruning hurts; lexical metric resolution is brittle; seed repetition at temperature 0 carries no information.

2. Related Work and Research Gaps

Neural and fine-tuned parsers. RAT-SQL encodes schema relations with relation-aware self-attention (57.2% exact match on Spider, 65.6% with BERT) [R1]. PICARD constrains decoding with incremental parsing to guarantee valid syntax [R2]; RESDSQL decouples schema linking from skeleton parsing [R3]; CodeS offers open 1B–15B models [R4]. These need labelled in-domain data, and syntactic validity does not imply correct semantics.

LLM prompting and agents. DIN-SQL decomposes the task into prompted modules (85.3% Spider test, 55.9% BIRD test) [R5]; DAIL-SQL shows that similarity-based example selection matters (86.6% on Spider) [R6]. Multi-agent and candidate-selection systems — MAC-SQL (59.59% BIRD test) [R7], CHESS (71.10%) [R8], CHASE-SQL (73.0%) [R9], XiYan-SQL (75.63% BIRD) [R10] — push academic scores highest at the price of many LLM calls. Fine-tuning with synthetic data and reinforcement learning (OmniSQL [R11], Arctic-Text2SQL-R1 [R12]) makes small models competitive.

Benchmarks. Spider [R13] (10,181 questions, 200 databases) is nearly saturated and has no nested data or business knowledge. BIRD [R14] (12,751 questions, 95 databases) adds large dirty databases and external-knowledge *evidence*; ChatGPT scored 40.08% versus 92.96% for humans. Spider 2.0 [R15] and ReFoRCE [R16] target enterprise workflows (35.83–36.56 best reported at the time). Dr.Spider [R17] measures robustness to 17 perturbation types, with drops of 14.0% on average and 50.7% for the hardest.

Semantic layers. Sequeda et al. [R18] report GPT-4 accuracy rising from 16% to 54% on enterprise SQL question answering when questions are posed over a knowledge-graph representation. Industrial products ground generation in metric definitions, but their internals are not published in the sources we examined.

2.1 Identified gaps

#	Gap	Evidence in the literature	Addressed here by
G1	Nested, heterogeneous event data with business metrics is almost unmeasured; benchmarks are flat or schema-clean	Spider/BIRD have no list/struct parameters; Spider 2.0 includes GA4 but is expensive and not metric-centred	EventSQL-Bench (§3.3)
G2	The *isolated* contribution of a semantic layer is rarely quantified with paired statistical tests	Evidence is mostly end-to-end system scores or a single before/after comparison [R18]	Cumulative and single-component ablations with McNemar tests (§4–5)
G3	Clarification on ambiguous questions is rarely evaluated as its own task	Benchmarks score only answerable questions	17 ambiguous questions, clarification P/R/F1
G4	Robustness of the *concept-resolution* step to paraphrase is untested; robustness work perturbs schema and questions generally [R17]	Dr.Spider perturbs inputs but has no metric-resolution stage	20 out-of-dictionary paraphrases (§5.5)
G5	Components such as schema pruning are assumed beneficial; their harm in modern LLM pipelines is seldom reported	Pruning is a standard stage in [R7], [R8]	Ablation on two benchmarks (§5.6)
G6	Most strong results use proprietary or very large models; local, privacy-preserving deployment is under-studied for this domain	Top BIRD systems use large API models	8B and 3B open-weight models run locally

3. Method

3.1 Pipeline

For a question *q*: (1) the **ambiguity detector** checks terms listed in the semantic layer with several candidate definitions and, if triggered, returns a clarification question without calling the LLM; (2) the **metric resolver** maps *q* to metrics via longest-match lookup over bilingual synonyms, expanding dependencies (AOV → revenue, orders), with an optional **LLM fallback** that selects metrics from the layer's catalogue when no synonym matches; (3) the **schema linker** (hand-implemented BM25) prunes tables and columns and an **event profiler** lists event names and parameter keys observed in the data; (4) the **prompt builder** adds metric definitions and three retrieved question–SQL examples (BM25 over a train pool); (5) the LLM generates SQL; (6) a **validator** accepts only SELECT/WITH statements and executes on a read-only connection; (7) on an error, empty or all-NULL result, the **corrector** feeds the SQL and message back for up to two rounds.

3.2 Semantic layer

A YAML file defines 15 metrics (revenue, item_revenue, ad_revenue, orders, aov, items_sold, dau, mau, paying_users, arppu, conversion_rate, cart_conversion_rate, cart_abandonment_rate,

tutorial_completion_rate, retention_d1). Each has a business description, Vietnamese and English synonyms and a canonical SQL expression. The layer also holds parameter-access guidance for `event_params` and `items`, and ambiguity rules (e.g., "revenue" → revenue / item_revenue / ad_revenue; "conversion rate" → conversion_rate / cart_conversion_rate).

3.3 EventSQL-Bench

A DuckDB database generated by a seeded stochastic process (seed 7): 1,500 users, 59 days from 2026-01-01, an `events` table with nested `event_params` (key/value structs with typed values) and `items`, and a `users` table. User activity decays over time and follows view → add-to-cart → checkout → purchase funnels, with tutorial, level-up and ad events.

Questions are generated from parameterised *families* in Vietnamese and English with executable gold SQL (non-empty, non-NULL on build). Four levels: L1 flat tables without metrics; L2 access to nested data; L3 business metrics; L4 joins, cohorts and multi-step logic. Of 162 questions, 37 form the few-shot pool and 125 are test items (108 answerable, 17 ambiguous with no gold SQL). Three families (D1 retention, cart abandonment, top categories; 12 test questions) are entirely absent from the pool. A further 20 paraphrases deliberately avoid dictionary synonyms (19 mapped to gold SQL).

3.4 Metrics and statistics

Execution accuracy (EX) compares result sets on the same database, allowing column permutation, requiring row order only when gold SQL has an outermost ORDER BY, and rounding floats to four decimals. Ambiguous questions are scored by clarification precision, recall and F1. Paired comparisons use the exact McNemar test on discordant questions; confidence intervals use the bootstrap. Gold-versus-gold evaluation yields 100% in unit tests (23 tests) and on a 30-question sample of both Spider and BIRD.

4. Experimental Setup

- **Models:** `qwen3:8b` (main) and `qwen2.5:3b`, served by Ollama on a laptop (Apple M1 Max, 32 GB), temperature 0, thinking disabled, 1,024 output tokens. API adapters (Anthropic, OpenAI) are implemented but **were not run** (no API access during this study).
- **Configurations:** baseline (DDL + question); +linking; +fewshot; +semantic; +correction; +clarify; plus single-component `semantic_only` and `correction_only`.
- **Languages:** Vietnamese (main) and English.
- **Public benchmarks:** fixed random samples of 200 questions (seed 123) from Spider dev and BIRD dev. For BIRD, `knowledge_only` supplies the benchmark's evidence without other components, as a proxy for a metric layer.
- **Seeds:** runs with seeds 0, 1, 2 were identical at temperature 0 (standard deviation 0), so repeated seeds are not used to estimate uncertainty.

5. Results

5.1 Ablation on EventSQL-Bench (Vietnamese, `qwen3:8b`, n = 108)

Configuration	EX	L1	L2	L3	L4	Withheld families (n = 12)
baseline	0.352	0.93	0.00	0.42	0.09	3/12
+linking	0.296	0.93	0.00	0.36	0.00	0/12
+fewshot	0.880	1.00	1.00	0.98	0.64	3/12
+semantic	0.963	1.00	1.00	0.98	0.91	10/12
+correction	0.981	1.00	1.00	0.98	0.97	12/12
+clarify	0.981	1.00	1.00	0.98	0.97	12/12
semantic_only	0.444	0.53	0.70	0.38	0.42	6/12
correction_only	0.370	0.93	0.00	0.46	0.09	3/12

Bootstrap 95% CIs: baseline [0.27, 0.44]; +fewshot [0.81, 0.94]; +semantic [0.93, 0.99]; +correction [0.95, 1.00]. McNemar: +linking vs baseline p = 0.031 (worse); +semantic vs +fewshot p = 0.0039; +correction vs +semantic p = 0.5; semantic_only vs baseline p = 0.19; correction_only vs baseline p = 0.5. On withheld families: +semantic vs +fewshot p = 0.0156; +correction vs +fewshot p = 0.0039.

Reading. Few-shot examples account for the largest jump, mostly through template reuse within a family (withheld families stay at 3/12). The semantic layer adds a significant further gain, concentrated where business definitions matter (L4) and where examples cannot help (withheld families). Self-correction helps little in isolation (0.352 → 0.370, n.s.) and adds 0.018 on top of the full stack (n.s.): it repairs visible errors but not silently wrong metric choices. Baseline failures are 43 schema errors, 22 wrong results, 5 runtime errors; Level 2 is 0% because the model does not know DuckDB's `list_filter` access pattern even with an event catalogue.

5.2 Clarification

On 17 ambiguous questions, clarification precision, recall and F1 are 1.00/1.00/1.00. Because detection is rule-based and the questions share a distribution with the rules, this is an upper bound, not an estimate of real-world performance.

5.3 Language and model size

Model	Language	baseline	+fewshot	+semantic	+correction
qwen3:8b	Vietnamese	0.352	0.880	0.963	0.981
qwen3:8b	English	0.333	0.889	0.991	not run
qwen2.5:3b	Vietnamese	0.167	0.556	0.556	0.657

The trend is the same in both languages. The 3B model benefits from the pipeline (0.167 → 0.657) but stays far below the 8B model and gains nothing from the semantic layer.

5.4 Public benchmarks (n = 200 each, `qwen3:8b`)

Configuration	Spider dev	BIRD dev
baseline	0.720	0.240
+linking	0.725	0.155
knowledge_only	not run	0.345

Configuration	Spider dev	BIRD dev
+knowledge	not run	0.220
+correction	not run	0.250

Evidence raises BIRD EX from 0.240 to 0.345, consistent with the value of explicit business knowledge (RQ2), although BIRD evidence is human-written per question and is not a reusable metric layer. The standard error at n = 200 is roughly three percentage points, and absolute numbers are not comparable to leaderboards.

5.5 Paraphrase robustness

Configuration	Metric-resolution recall	End-to-end EX
no semantic layer	0/20	11/19 (0.58)
semantic layer, lexical resolver	0/20	10/19 (0.53)
semantic layer + LLM fallback	17/20	17/19 (0.89)

The lexical resolver recalled none of the 20 paraphrases, whereas it matched every one of the 162 original questions; the latter is therefore an upper bound created by authoring synonyms alongside questions. The LLM fallback restores the benefit of the layer. With 19 items, the 11 vs 10 difference should not be interpreted.

5.6 Schema pruning

BM25 pruning reduced EX on both EventSQL-Bench (0.352 → 0.296, p = 0.031) and BIRD (0.240 → 0.155) and was neutral on Spider (0.720 → 0.725). We observed ambiguous columns in joins and misuse of `event_params[?]` as a prepared-statement placeholder, but we did **not** verify the causal mechanism.

5.7 Answers to the research questions

- **RQ1:** weakly supported. Gains come from retrieval and the semantic layer; execution-feedback correction is not significant alone.
- **RQ2:** supported on this benchmark (+0.083 EX over few-shot, p = 0.0039; 3/12 → 10/12 on withheld families), conditional on a fallback resolver for paraphrase.
- **RQ3:** partially answered. An 8B local model reaches 0.963–0.981 in-domain, a 3B model 0.657; no comparison with API models was possible.

6. Discussion

Why the semantic layer helps. Without it, the model must infer both nested-access syntax and business definitions; few-shot examples can supply the former for seen templates but cannot supply the latter for new ones. A metric layer converts an open-ended generation problem into definition retrieval plus assembly, which is also easier to audit and to maintain: a definition is fixed once and reused.

Why correction helps less than expected. Execution feedback detects errors the database exposes (unknown column, syntax). The most damaging errors in this domain are silently wrong

metrics that execute and return plausible numbers; only an explicit definition prevents these.

Practical implication. For event-analytics deployments, effort invested in curated metric definitions and in a robust (LLM-assisted) resolver is likely to pay off more than extra agent iterations; small local models are usable when the pipeline supplies definitions and examples.

Threats to validity.

1. *Same-author bias:* gold SQL and semantic-layer SQL were written by the same person and are near-identical, so the semantic-layer gain is optimistic.
2. *Template leakage:* few-shot examples share templates with test items of the same family.
3. *Small withheld set* (n = 12) and small paraphrase set (n = 20).
4. *Synthetic, single database*; EX on one database rather than test-suite accuracy, so coincidental matches may inflate scores.
5. *Possible contamination* of Spider/BIRD in model training data.
6. *No variance estimate across seeds* (deterministic decoding) and no sampling-based repeats.
7. *Rule-based clarification* evaluated in-distribution.
8. *No API-model comparison*, so claims about relative performance to larger models are not made.

7. Conclusion

On nested event data, explicit metric definitions contributed more than execution-feedback agent loops: with a locally run 8B model, a semantic layer lifted EX from 0.880 to 0.963 over few-shot prompting ($p = 0.0039$) and generalised to question families absent from the examples, while a purely lexical resolver proved brittle and an LLM fallback repaired it. BM25 schema pruning was harmful. Future work includes independent annotation, test-suite evaluation, API-model comparison, learned (embedding) schema linking, majority-vote candidates, and evaluation on real GA4 data (Spider 2.0).

Data and code availability

Code, benchmark generator, semantic layer, question set, run logs and analysis scripts are in the project repository (`querycraft-ai`); all numbers in this paper come from `experiments/results_main.md`, `experiments/results_paraphrase.md` and `experiments/run_remaining.log`. [Thay bằng URL công khai/DOI Zenodo khi nộp.]

References

> Chi tiết thư mục (tác giả, năm, tạp chí/hội nghị) cần đối chiếu lại trên arXiv/nhà xuất bản trước khi nộp; ở đây chỉ ghi mã arXiv đã xác minh từ abstract.

[R1] RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers, arXiv:1911.04942.
[R2] PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models, arXiv:2109.05093.
[R3] RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL,

arXiv:2302.05965.
[R4] CodeS: Towards Building Open-source Language Models for Text-to-SQL, arXiv:2402.16347.
[R5] DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction, arXiv:2304.11015.
[R6] DAIL-SQL: Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation, arXiv:2308.15363.
[R7] MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL, arXiv:2312.11242.
[R8] CHESS: Contextual Harnessing for Efficient SQL Synthesis, arXiv:2405.16755.
[R9] CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL, arXiv:2410.01943.
[R10] XiYan-SQL: A Multi-Generator Ensemble Framework for Text-to-SQL, arXiv:2411.08599.
[R11] OmniSQL: Synthesizing High-quality Text-to-SQL Data at Scale, arXiv:2503.02240.
[R12] Arctic-Text2SQL-R1: Simple Rewards, Strong Reasoning in Text-to-SQL, arXiv:2505.20315.
[R13] Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task, Yu et al., EMNLP 2018.
[R14] BIRD: Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs, arXiv:2305.03111.
[R15] Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows, arXiv:2411.07763.
[R16] ReFoRCE: A Text-to-SQL Agent with Self-Refinement, Format Restriction, and Column Exploration, arXiv:2502.00675.
[R17] Dr.Spider: A Diagnostic Evaluation Benchmark towards Text-to-SQL Robustness, 2023.
[R18] A Benchmark to Understand the Role of Knowledge Graphs on Large Language Model's Accuracy for Question Answering on Enterprise SQL Databases, arXiv:2311.07509.

Việc cần làm trước khi nộp (đọc kỹ)

Bản nháp này trung thực với số liệu hiện có, nhưng để nộp tạp chí Q3/Q4 cần bổ sung:

1. **Chọn tạp chí và kiểm tra quartile** trên Scimago/JCR của năm hiện tại, đọc scope và định dạng (mình chưa tra cứu tạp chí cụ thể, nên chưa gợi ý tên).
2. **Người thứ hai viết/kiểm SQL vàng** cho một tập con (hoặc đo inter-annotator agreement) để giải quyết thiên lệch cùng tác giả (hạn chế lớn nhất).
3. **Chạy ít nhất một model API** (cần key) để trả lời RQ3; nếu không, giữ cách diễn đạt hiện tại và bỏ so sánh với model lớn.
4. **Tăng số câu paraphrase và unseen** (hiện 20 và 12), vì reviewer sẽ phản đối n quá nhỏ.
5. **Chạy lại ≥ 1 cấu hình với temperature > 0 và nhiều seed** để có phương sai thật; thêm test-suite accuracy nếu được.
6. **Điều tra nguyên nhân schema linking làm tệ hơn** (phân tích 20–30 ca lỗi), vì đây là phát hiện mới nhưng hiện chưa có giải thích đã kiểm chứng.
7. **Đối chiếu tài liệu tham khảo** (tác giả, năm, nơi xuất bản), thêm các công trình nên trích như Metabot/MetricFlow dưới dạng tài liệu kỹ thuật, và thêm công trình hỏi lại khi mơ hồ nếu tìm được.
8. Bài viết bằng tiếng Anh; nếu tạp chí yêu cầu mẫu riêng (LaTeX/Word), mình có thể chuyển định dạng.